TECHNICAL WHITE PAPER

Face Recognition-Based Classroom Attendance System

Using Multi-Angle Facial Enrollment and Liveness Verification

Submitted by	PRASANSHA PRIYA ANCHAL KUMARI AASHI KAUR BHOOMI LADIA UPASANA MAJUMDER
Roll / Registration No.	2461073 2461075 2461076 2461077 2461078
Department	Computer Science and Engineering (AI & ML)
Institution	Heritage Institute of Technology, Kolkata
Course / Subject	Image Processing

Academic Project Report • October 2026

Abstract

Manual classroom attendance consumes instructional time and can be affected by proxy attendance, recording errors, and repetitive administrative work. This project describes a computer-vision-based attendance system that uses facial recognition, multi-angle enrollment, image-quality checks, temporal identity consistency, and a randomized head-movement challenge. The system is organized as a Next.js and TypeScript web application, a Python 3.11 FastAPI machine-learning service, and a PostgreSQL 16 database extended with pgvector for vector similarity operations.

During enrollment, a student provides explicit consent and follows guided front, left, right, up, and down head poses. The service validates incoming frames, extracts facial embeddings using the InsightFace buffalo_l model pack (including ArcFace-based recognition), and stores accepted embeddings rather than source face photographs. During attendance, the system compares embeddings against enrolled representations and requires a configured similarity threshold, a margin over the runner-up, repeated identity consistency across frames, and a server-verified head-movement challenge before marking a student present. A unique database constraint prevents duplicate attendance entries for a student in the same session.

The design aims to improve the efficiency and auditability of classroom attendance while limiting the storage of raw biometric imagery. Its limitations include vulnerability to some replay-video attacks, the need for threshold calibration with representative classroom data, and the operational complexity of scaling in-memory enrollment and liveness state. No quantitative accuracy result is asserted in this report because measured test data was not provided; evaluation metrics and a validation plan are therefore specified for future testing.

Keywords: face recognition, image processing, facial embeddings, ArcFace, RetinaFace, liveness verification, computer vision, classroom attendance, pgvector.

1. Introduction

Attendance is a routine but important part of classroom administration. Manual roll calls and paper-based records can take time, are difficult to aggregate, and may be vulnerable to proxy attendance or transcription errors. Face recognition offers a contactless method for associating a visible face with an enrolled identity. However, a reliable attendance system must do more than compare a single image: it must handle variation in pose and lighting, reject poor-quality frames, manage uncertain matches, discourage presentation attacks, and protect sensitive biometric information.

The proposed system combines a browser-based user experience with a dedicated Python service for face analysis and a persistent database. The browser provides guided enrollment and kiosk capture. The backend performs quality validation, embedding extraction, similarity matching, and server-side verification of a randomized head movement. The database stores the resulting attendance records and associated operational data.

The project is best understood as an applied image-processing and computer-vision system with a deep-learning recognition component. Some stages use neural-network models, while others use conventional numerical checks, rule-based decisions, database constraints, and application logic.

2. Problem Statement

The problem is to design a classroom attendance workflow that identifies enrolled students from live webcam frames while reducing false matches, duplicate marks, and straightforward attempts to use a static photograph. The system must support enrollment from multiple head poses, give staff a way to handle unrecognized students, and provide attendance reports without retaining raw enrollment photographs.

2.1 Objectives

- Build a guided enrollment workflow that collects multiple facial views after explicit consent.
- Use face detection and learned facial embeddings to compare a live face with enrolled identities.
- Reject unsuitable frames using quality and pose checks before accepting a sample or recognition result.

- Require a stable identity match and a randomized, server-verified head-movement challenge before automatic attendance marking.
- Prevent duplicate attendance records for the same student and session.
- Provide teacher and administrator tools for classes, sessions, student records, staff accounts, manual marking, and reports.
- Minimize stored imagery by retaining embeddings instead of face photographs, while supporting deletion of face data.
- Define a repeatable evaluation plan for recognition quality, usability, latency, and presentation-attack resistance.

3. System Overview and Architecture

The system is split into three main layers. The web layer captures user interaction and provides visual guidance; the ML/API layer performs the computationally intensive biometric pipeline; and the database layer persists identities, embeddings, consent records, sessions, attendance, and recognition scores.

Layer	Technology	Responsibilities
Web application	Next.js, TypeScript	Enrollment UI, webcam capture, MediaPipe Face Landmarker pose guidance, attendance kiosk, staff dashboard and CSV export.
ML and API service	Python 3.11, FastAPI, InsightFace	Frame validation, face detection, embedding extraction, recognition, liveness challenge checks, authentication-protected API endpoints.
Database	PostgreSQL 16, pgvector	Persistent relational records and vector similarity operations over facial embeddings.
Runtime / tooling	Docker Compose, uv, Uvicorn	Local database orchestration, Python dependency management, and ASGI application serving.

3.1 High-Level Data Flow

Stage	Description
1. Webcam	Browser obtains a frame from the camera.
2. Pose guidance	MediaPipe landmarks guide the user toward a target head pose.
3. API request	The browser sends a frame to the FastAPI service over the configured origin.
4. Quality gate	The service validates face count, size, blur, brightness, and pose.
5. Embedding	InsightFace generates a numerical face representation.
6. Match / challenge	The service compares candidate embeddings and validates temporal and liveness conditions.
7. Database	Accepted embeddings or attendance events are persisted with applicable constraints.
8. UI response	The browser displays success, rejection, or manual-mark guidance.

Important separation of responsibilities
MediaPipe Face Landmarker is described as running in the browser to guide head pose. InsightFace runs in the Python service for face analysis and recognition. These components support different stages and should not be described as interchangeable.

4. Technologies and Python Components

4.1 Python and FastAPI

Python 3.11 is the implementation language for the machine-learning and API service. FastAPI defines HTTP endpoints and request/response handling, allowing the frontend to communicate with the recognition pipeline. Uvicorn serves the FastAPI application as an ASGI server. Pydantic is commonly used by FastAPI for data validation; the exact request models should be confirmed against the source code.

4.2 InsightFace, RetinaFace, and ArcFace

InsightFace is the face-analysis framework used by the backend. The configured buffalo_l model pack is described as combining RetinaFace for face detection with ArcFace-based recognition. RetinaFace locates faces and provides facial localization information. ArcFace is a deep-learning approach designed to produce discriminative identity embeddings by applying an additive angular-margin objective during training. The detector and recognizer have separate roles: detecting a face does not by itself identify the person.

4.3 MediaPipe Face Landmarker

MediaPipe Face Landmarker runs in the browser and estimates facial landmarks that help the enrollment interface guide a person through target head poses. The user interface can use landmark geometry and pose estimates to indicate whether the face is sufficiently aligned and stable for capture. In the described design, this is a capture-guidance component; final server-side validation remains important because client-side checks alone should not be trusted as proof of identity or liveness.

4.4 PostgreSQL and pgvector

PostgreSQL stores the application's relational data, including student and staff records, sessions, consent metadata, attendance entries, and recognition logs. pgvector adds vector data types and similarity-search operations to PostgreSQL. This enables the application to search or compare stored facial embeddings without using a separate vector database. Exact indexing strategy, distance operator, and query implementation should be documented from the repository code.

4.5 Supporting tools and dependencies

Component	Role	Classification
NumPy (if imported)	Array and vector operations, including numerical similarity calculations where implemented.	Python numerical library; verify imports.
Pydantic (if used directly)	Validates request and response schemas.	Data validation; verify imports.
JWT library	Creates and verifies staff authentication tokens.	Security dependency; exact package name must be checked in dependency files.
uv	Installs and runs pinned Python dependencies.	Package/environment tool, not an image-processing algorithm.
Uvicorn	Runs the FastAPI ASGI application.	Application server.
Docker Compose	Starts the local PostgreSQL and pgvector service.	Container orchestration.

The project description identifies the high-level components above, but a final implementation inventory should be generated from pyproject.toml/uv.lock and the Python import statements. A library should not be claimed as used merely because it is common in similar projects.

5. Image Processing and Recognition Methodology

5.1 Image acquisition

A webcam supplies frames to the browser. The attendance kiosk is designed to send a frame approximately every 400 milliseconds while a face is visible. Image acquisition is affected by camera resolution, distance, lighting, motion, compression, and camera placement. These factors can influence the quality of subsequent detection and recognition.

5.2 Face detection

Face detection identifies a face region in an image and estimates its location. The configured InsightFace pipeline uses a RetinaFace detector. The detector's output allows downstream processing to focus on the face region and associated

facial keypoints. The system rejects frames with no suitable face or with more than one face, consistent with its one-person-at-a-time kiosk design.

5.3 Image quality gate

Before a sample is enrolled or a recognition result is accepted, the backend checks that the frame meets configurable quality conditions. The documented checks include blur, brightness, face size, number of detected faces, and pose consistency. These checks are quality-control gates rather than identity classifiers.

Check	Purpose	Failure consequence
Blur / sharpness	Avoid feature extraction from motion-blurred or defocused frames.	Reject frame and request another capture.
Brightness	Avoid underexposed or overexposed face regions.	Reject frame if outside configured bounds.
Face size	Ensure the face occupies enough pixels for meaningful analysis.	Reject frame if too small.
Face count	Maintain one-person-at-a-time enrollment and kiosk assumptions.	Reject frame when multiple faces are detected.
Pose consistency	Ensure the captured frame matches the requested enrollment pose.	Reject frame or request repositioning.
Cross-sample consistency	Check that a new enrollment sample is compatible with earlier samples for the claimed identity.	Reject inconsistent enrollment sample.

5.4 Facial feature extraction and embeddings

After a face passes the relevant gates, the recognition model converts its appearance into a fixed-length numerical embedding. The described system stores 512-dimensional embeddings. If e denotes an embedding, it can be represented as $e = [e_1, e_2, \ldots, e_{512}]$. These values encode learned facial features in a space where images of the same identity are expected to be closer than images of different identities, subject to model limitations and capture conditions.

The system retains multiple embeddings per pose rather than replacing all samples with one average. This preserves sample-level variation that may be useful when the person is later seen under a different pose or condition. The report's final description of the storage layout should be checked against the actual database schema.

5.5 Similarity matching

At attendance time, the new embedding is compared with enrolled embeddings. A common metric for normalized embeddings is cosine similarity:

$$\cos(e_1, e_2) = (e_1 \cdot e_2) / (||e_1|| \, ||e_2||)$$

The exact metric used by this project must be verified in the matching code. A higher cosine similarity usually indicates greater embedding similarity, but it is not a calibrated probability. The best candidate must satisfy MATCH_THRESHOLD and exceed the runner-up score by at least MATCH_MARGIN. Requiring both conditions helps reject low-confidence and ambiguous matches.

5.6 Temporal consistency

A single frame may be noisy or ambiguous. The system therefore requires the same identity to remain the best match for STABLE_FRAMES consecutive frames; the supplied configuration describes a value of three. Temporal consistency reduces the chance that a transient detection or one-frame mismatch immediately creates an attendance record. It does not, by itself, establish liveness.

5.7 Threshold calibration

Recognition thresholds should be tuned using representative data from the intended classroom. The repository describes a calibration script that compares genuine-match scores with impostor-match scores and suggests a threshold. Genuine comparisons are pairs from the same enrolled person; impostor comparisons are pairs from different people. The

selected threshold and margin should balance false acceptance against false rejection and should be re-evaluated when cameras, lighting, enrollment procedures, or the student population change.

6. Detailed Workflows

6.1 Student enrollment workflow

1. The student enters identity details and gives explicit consent. Consent time and version are recorded.
2. The browser opens the camera and guides the student through front, left, right, up, and down poses.
3. MediaPipe landmarks guide the head toward each target pose and help determine when a frame is stable enough to capture.
4. The backend detects the face and checks blur, brightness, face size, face count, and pose consistency.
5. The recognition pipeline extracts an embedding from each accepted frame.
6. The service checks that samples are consistent with the claimed identity's earlier samples and runs a duplicate-face check against other enrolled identities.
7. Accepted embeddings are stored with the corresponding student and pose. The design describes approximately 20 embeddings per pose.
8. The UI reports the enrollment outcome and allows rejected poses to be recaptured.

6.2 Attendance workflow

9. A teacher creates or opens an attendance session for a class.
10. The kiosk captures frames while a face is visible and sends them to the API at approximately 400 ms intervals.
11. The backend validates image quality, detects the face, and computes an embedding.
12. The embedding is compared with enrolled samples; the best score must meet MATCH_THRESHOLD and exceed the runner-up by MATCH_MARGIN.
13. The same identity must remain stable for STABLE_FRAMES consecutive frames.
14. The backend checks for a frontal baseline followed by the randomly selected head movement challenge.
15. If identity, stability, and challenge checks pass, the attendance entry is saved.
16. If recognition does not pass, the kiosk shows a not-recognized message and a teacher can use the manual-mark fallback.

6.3 Liveness challenge workflow

The backend selects a movement challenge from the supported actions (turn left, turn right, or look up). It checks face keypoints across frames for a frontal baseline and the required movement sequence. Randomizing the action makes a simple static photo less likely to pass. Because a video replay may contain a matching movement, the mechanism should be described as a basic active liveness challenge, not as complete anti-spoofing protection.

6.4 Teacher onboarding and administration

Teachers can sign up and wait for administrator approval, or an administrator can invite staff. The described first-sign-up behavior makes the first registrant on a fresh installation the administrator; this bootstrap behavior should be protected operationally and disabled or constrained once initial setup is complete. The application also supports password setup, a guided tour, camera and lighting checks, and a setup checklist covering class creation, enrollment, session scheduling, and attendance.

6.5 Reporting and record management

The dashboard supports roster viewing, manual attendance marking and undo, student re-enrollment, deletion of face data, session reports, attendance percentages, and CSV export. Administrative controls manage staff accounts. A database uniqueness constraint on the pair (user_id, session_id) prevents duplicate attendance marks for the same person and session.

7. Database, API, and Security Design

7.1 Data persistence

The system uses PostgreSQL 16 with pgvector. The described data includes student/staff identity records, per-pose facial embeddings, consent metadata, class and session records, attendance entries, and recognition scores in a recognition_log. The exact table names, foreign keys, deletion rules, and vector indexes should be documented from the database models and migrations.

7.2 API access control

The project states that all endpoints except login require a staff token, and that face-data deletion and staff management are administrator-only operations. JWT-based authentication is part of the described design. Production deployment should use a strong secret, short and appropriate token lifetimes, secure password handling, TLS, restrictive CORS settings, and audit logging for sensitive administrative actions.

7.3 Privacy safeguards

- Explicit consent is required before facial capture, with consent timestamp and version recorded.
- The design stores facial embeddings rather than source face photographs.
- Administrators can delete face data while preserving attendance history, or remove a student according to application behavior.
- Recognition logs are described as recording scores rather than images.
- Production guidance includes TLS and encryption at rest for the database volume or disk.
- Access to biometric data and administrative operations should be limited to authorized roles and periodically reviewed.

Privacy and legal scope

The project describes safeguards intended to support privacy-conscious handling of biometric data. This report does not certify compliance with the Digital Personal Data Protection Act or any other law. A real deployment requires a legal and institutional review, clear notice and consent practices, retention policies, access controls, and a process for handling data-subject requests.

8. Evaluation Plan and Metrics

A defensible research report should distinguish implemented features from measured performance. Since no test dataset, measured accuracy, or latency results were supplied, this report does not invent numerical results. The following plan can be used to evaluate the implementation.

Metric	Definition / purpose	Suggested test
True acceptance / genuine match rate	Proportion of genuine attempts correctly accepted.	Collect repeated attempts from enrolled participants under varied poses and lighting.
False acceptance rate (FAR)	Proportion of impostor attempts incorrectly accepted.	Test non-enrolled people and cross-identity comparisons.
False rejection rate (FRR)	Proportion of genuine attempts incorrectly rejected.	Repeat genuine attempts across realistic classroom conditions.
Precision of attendance marks	Share of automatically marked entries that belong to the correct student.	Manually audit a labeled sample of sessions.
Recognition latency	Time from frame submission to recognition decision.	Measure median and 95th-percentile API latency under stated hardware and load.
Enrollment completion rate	Share of enrollment attempts that complete all required poses.	Record completed versus abandoned/rejected enrollment flows.
Liveness challenge pass/fail	Correct behavior on live users and tested presentation attacks.	Test static photos and controlled replay videos; report each attack type separately.

Operational reliability	Errors, timeouts, duplicate attempts, and recovery behavior.	Run repeated sessions and inspect logs and database consistency.

8.1 Experimental protocol

17. Use informed, voluntary participants and an approved process for collecting any evaluation data.
18. Separate enrollment samples from evaluation samples to avoid reporting performance on the same frames used for enrollment.
19. Include frontal and angled views, different lighting levels, different distances, and common appearance changes such as glasses where feasible.
20. Measure genuine and impostor score distributions before choosing MATCH_THRESHOLD and MATCH_MARGIN.
21. Record hardware, camera, model pack, software versions, image conditions, sample counts, and the exact configuration.
22. Report the number of attempts as well as percentages, and avoid generalizing beyond the tested environment.

8.2 Results table template

Experiment	Sample count	Result	Notes
Genuine identity attempts	[To be measured]	[To be measured]	Include pose and lighting breakdown.
Impostor identity attempts	[To be measured]	[To be measured]	Report false acceptances.
Static-photo challenge tests	[To be measured]	[To be measured]	State device, distance, and challenge configuration.
Replay-video challenge tests	[To be measured]	[To be measured]	Known limitation; report successful attacks.
API latency	[To be measured]	[To be measured]	Report median and 95th percentile.
Enrollment completion	[To be measured]	[To be measured]	Record rejected samples and retries.

9. Limitations and Future Enhancements

9.1 Known limitations

- A randomized head-turn challenge can deter static photographs but may not stop a replayed video that performs the required movement.
- Recognition quality depends on enrollment coverage, lighting, camera quality, head pose, and threshold selection.
- A single threshold may not yield the same error rates across all groups and operating conditions; evaluation must consider performance variation.
- The kiosk is designed for one person at a time; simultaneous whole-class capture is outside the current design.
- Enrollment drafts and liveness attempts are described as in-memory state. Multiple API workers may not share this state; scaling requires shared state such as Redis or another coordinated store.
- The first-user-becomes-admin bootstrap rule can be risky if exposed in an uncontrolled deployment.
- Embeddings are biometric data and remain sensitive even when original face photographs are not retained.

9.2 Future enhancements

- Evaluate a passive face anti-spoofing model, such as a MiniFASNet-based model, as an additional signal rather than treating it as a guaranteed solution.
- Move transient enrollment and liveness state into a shared store before deploying multiple workers.
- Add role-based audit trails, retention automation, backup verification, key rotation, and incident-response procedures.
- Evaluate threshold calibration on representative, consented classroom data and monitor false acceptance and false rejection rates.
- Add resilience to camera failure, API timeouts, network interruption, and offline attendance contingencies.

- Explore multi-person capture only after validating detection, identity association, privacy, and fairness under crowded classroom conditions.

10. Deployment and Reproducibility

The described local development environment uses Docker Compose to start PostgreSQL 16 with pgvector. The ML service is run from the ml-service directory with uv-managed dependencies and Uvicorn. On first startup, the buffalo_l model pack is downloaded (the project description estimates approximately 280 MB). The web application runs from the web directory with npm scripts.

Stage	Illustrative command / action	Purpose
Database	docker compose up -d	Start PostgreSQL and pgvector services.
ML environment	cd ml-service; cp .env.example .env; uv sync	Configure environment variables and install locked dependencies.
Bootstrap admin	uv run python -m app.cli create-staff ...	Create an initial administrator using the CLI documented by the repository.
API server	uv run uvicorn app.main:app --port 8000	Start the FastAPI service for local development.
Web environment	cd web; cp .env.local.example .env.local; npm install; npm run dev	Install frontend dependencies and start Next.js.

These commands are adapted from the supplied project notes and may need shell-specific formatting. Configure JWT_SECRET as a long random secret, and ensure NEXT_PUBLIC_API_URL and CORS_ORIGINS match the intended service origins. Browser camera access is generally restricted to secure contexts such as HTTPS or localhost. A LAN kiosk should use HTTPS or a TLS reverse proxy. Production deployment also requires secure secrets, TLS, database backups, tested restore procedures, encryption at rest, dependency updates, monitoring, and controlled access.

Reproducibility note
Record exact model and dependency versions, environment variables (without secrets), database migration version, camera configuration, threshold values, and calibration dataset description with any reported experiment.

11. Conclusion

This project presents a layered approach to classroom attendance using facial recognition and image processing. Multi-angle enrollment supplies samples from different head poses; quality gates reject unsuitable frames; InsightFace generates facial embeddings; similarity threshold and runner-up margin checks constrain identity matching; temporal consistency and a randomized head-movement challenge add safeguards before attendance is recorded. PostgreSQL and pgvector support persistent records and embedding comparison, while the web dashboard provides enrollment, classroom sessions, manual fallback, and reporting.

The architecture addresses several practical failure modes that a simple one-frame face matcher would not handle. Nevertheless, liveness verification is not equivalent to comprehensive anti-spoofing, and recognition performance must be measured rather than assumed. Future work should prioritize controlled evaluation, calibrated thresholds, replay-attack testing, secure deployment, shared state for scaling, and responsible handling of biometric information.

References

23. [1] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019. DOI: 10.1109/CVPR.2019.00482.
24. [2] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-stage Dense Face Localisation in the Wild,” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

25. [3] InsightFace, “InsightFace: 2D and 3D Face Analysis Project,” project documentation and model information. https://github.com/deepinsight/insightface
26. [4] Google AI Edge, “MediaPipe Face Landmarker,” official documentation. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker
27. [5] FastAPI, “FastAPI Documentation.” https://fastapi.tiangolo.com/
28. [6] pgvector, “Open-source vector similarity search for Postgres,” project documentation. https://github.com/pgvector/pgvector
29. [7] PostgreSQL Global Development Group, “PostgreSQL Documentation.” https://www.postgresql.org/docs/
30. [8] Uvicorn, “Uvicorn documentation.” https://www.uvicorn.org/
31. [9] Python Packaging Authority, “uv documentation.” https://docs.astral.sh/uv/
32. [10] Government of India, “Digital Personal Data Protection Act, 2023,” official legislative text. The applicable legal position and compliance obligations should be checked against current rules and institutional guidance.

Appendix A. Configuration Parameters to Document

Parameter / area	Purpose	Report what was actually configured
MATCH_THRESHOLD	Minimum similarity score for the best identity candidate.	[Value from environment/config]
MATCH_MARGIN	Required gap between the best and runner-up scores.	[Value from environment/config]
STABLE_FRAMES	Consecutive frames required for identity consistency.	3 according to supplied project description; verify config.
Blur and brightness limits	Reject low-quality frames.	[Values and measurement method]
Pose deltas / tolerances	Determine acceptable target pose and movement.	[Values and coordinate conventions]
Session grace window	Control when attendance may be recorded for a session.	[Configured duration]
Re-enrollment trigger	Identify students whose recent match confidence falls.	[Rule and evaluation window]
Database vector index	Affects embedding search performance and recall.	[Index type, distance operator, if any]

Appendix B. Implementation Details to Verify Before Submission

- Confirm exact Python dependencies and versions from pyproject.toml and uv.lock.
- Confirm the actual similarity metric and vector-search operator in the recognition code.
- Confirm how the system checks pose and head movement server-side, including landmark definitions and thresholds.
- Confirm database table names, constraints, cascade behavior, and how consent is stored.
- Confirm that no face images or temporary image files are persisted in logs, caches, object storage, or debugging output.
- Run the calibration script and insert measured results only after collecting appropriate evaluation data.
- Check the current repository deployment guide before using any command or making a production-readiness claim.